THE LAWS OF SOFTWARE TEAMS

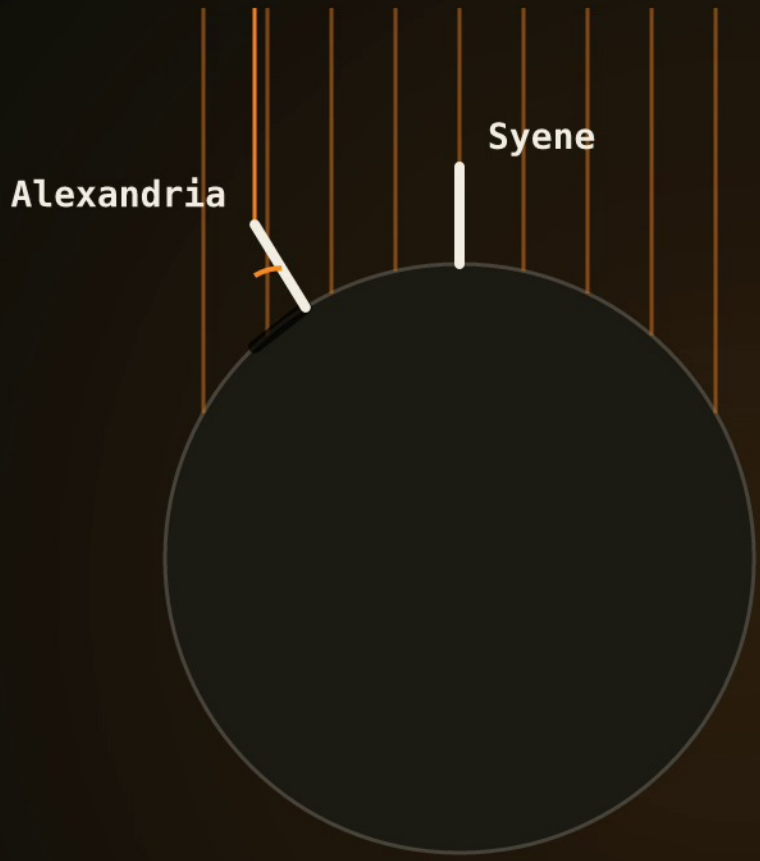
Gilb's law

Anything you need to quantify can be measured better than not at all

swipe →

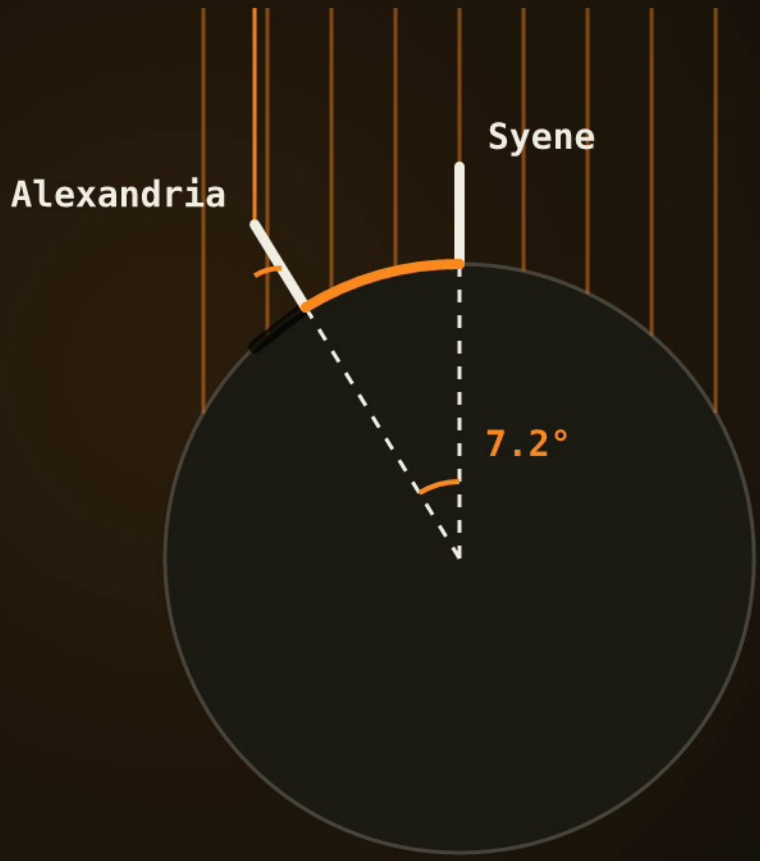
Ivan Rykov

He measured the Earth with a stick



~2,200 years ago. No satellites

Noon. No shadow in Syene 7.2° in Alexandria



7.2° = 1/50 of a circle

5,000 stadia × 50 = 250,000

"Anything you need to quantify can be measured in some way that is superior to not measuring it at all"

Gilb's law

Tom Gilb · named in Peopleware, DeMarco & Lister



Measuring ≠ certainty
It's less uncertainty

How long until a new hire ships?

weeks · example



first guess: 1-20 weeks

after 5 answers: 4-7 weeks

Illustrative example · idea from Douglas W. Hubbard

You need less data than you think



Pick 5 at random

94% chance the median is inside their range

Hubbard's Rule of Five · $1 - 2 \times (½)^5 = 93.75\%$

Two laws, one balance

GOODHART

Don't make it a target

GILB

Don't stop measuring

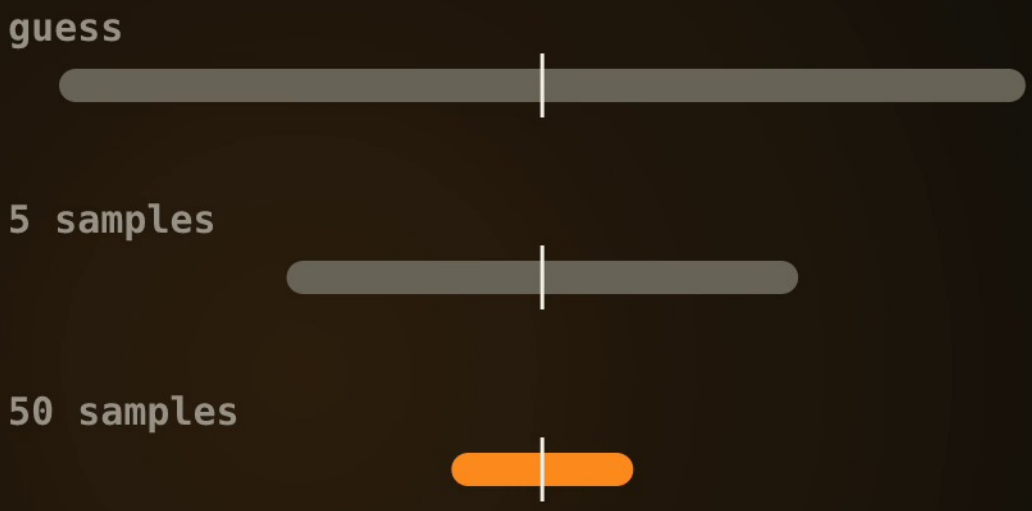
No data is worse

Start small

1. **Name the decision**
   the number should change
2. **Guess a range**
   e.g. 2-12 weeks
3. **Sample a little**
   and watch the range shrink

Hubbard: if it can't change a decision,
it isn't worth measuring

A rough number beats a confident guess



HOW TO USE IT

1. **Measure only what leads to a decision. A metric that changes nothing is noise. I have seen usage data show that a problem the team planned to solve did not exist. Delete a dashboard that nobody opens, or explain why it exists**
2. **Set a target only when it comes from a problem, a requirement or a request. A target that comes from nowhere wastes the team's time. I have seen a team tune a service for a speed that nobody asked for**
3. **Keep five to seven key metrics. Treat the rest as supporting signals**

HOW TO USE IT

4. Do not turn tasks, pull requests or activity into a score for one person. They are signals, and a person does much more work than these numbers show
5. Give rough numbers to things that seem hard to measure, such as tech debt or team mood. If there is no data yet, collect it every week
6. Break the numbers down: by cost, by usage, by importance. A cost breakdown shows what to optimise first. A usage breakdown shows what nobody uses

HOW TO USE IT

7. **Treat 1:1s as data. Record the calls if people agree. If not, write notes by hand. Keep them private and look for problems that come up again**
8. **Run surveys rarely, with a clear goal and specific questions**

When the team uses AI tools, measure for the team: the number and size of pull requests, review time, time to merge, later fixes and rollbacks. When review becomes the bottleneck, add automatic review and more checks to help reviewers

Sources: DeMarco and Lister, Peopleware · Douglas W. Hubbard, How to Measure Anything · Eratosthenes

next: Goodhart's law